JavaScript 1.8

Memory Management

Print-first interview refresher | reachability → ownership → evidence

Ask two questions:
who still holds it?
when should ownership end?

1 | Follow strong paths from roots

Long-lived source
window / timer / registry

→

Callback + closure
access to captured bindings

→

Object graph
state → DOM / rows / buffers

null or delete removes one edge. Other aliases may keep the same graph reachable.

An unreachable cycle may be collected. GC timing is not guaranteed; scope is not lifetime.

2 | Close the owner, then release its resources

Only remove the panel

window → resize handler → detached node

The external path still exists.

Finish the lifecycle

Remove handler → cancel → unsubscribe

Drop heavy links in saved disposers too.

Match event target, type, function identity and capture. A new arrow is a different function.
once waits for an event; abort only helps APIs that support its signal.

3 | Cache and closure: intentional retention needs a policy

Closure: capture the data you need; do not assume every local is physically retained.

Cache: set an entry / byte budget and eviction or cleanup policy. Entry count is not byte size.

Lazy TTL alone can leave old, unread keys stored indefinitely.

WeakMap may fit object-key metadata. WeakRef and finalizers do not replace required cleanup.

4 | Blob URL: the browser owns a registration

Create
URL.createObjectURL(blob)

→

Use
keep valid for all consumers

→

Finish
URL.revokeObjectURL(url)

Replace: revoke the old URL. Dispose: revoke the current URL. Forgetting the string is not cleanup.
Do not revoke while the consumer still needs the URL; no universal zero-delay timer.

5 | Classify the memory symptom before fixing it

Peak
Temporary simultaneous copies

Leak
Unneeded data stays reachable

Bloat / warm-up
Useful data, check the budget

Allocation churn
Short-lived objects, frequent GC

6 | Compare the same completed scenario

Warm up → baseline → repeat open / use / close → compare snapshots → inspect retainers

Find the owner → remove the unwanted path → repeat in a fresh comparable state.

Shallow size: the object itself. Retained size: what would become unreachable without it.

Console references can affect the result. One high process-memory reading does not prove a leak.

Test the contract, observe the collector

Check cleanup, repeat calls and partial failure. Do not assert GC deadlines, finalizers or exact freed bytes.